

Where Accountability Lives: Mapping Human Responsibility to Workflow Artifacts in Agentic Software Development

Sabry E. Farrag 

School of Architecture, Computing and Engineering
University of East London, London, United Kingdom
sabryelgatem@gmail.com

Abstract

Coding agents now author commits, open pull requests, and push code in production repositories. Who is accountable for that work is settled in two places that do not refer to each other: the platform controls that gate what an agent may do, and the provider terms that allocate responsibility for what it produces.

We examine both. Across four widely used agentic coding tools and eighteen governing policy documents, we map responsibility onto the workflow events that produce observable artifacts: task assignment, plan approval, commit authorship, pull request creation, check execution, review, approval, merge, and deployment. For each event we ask who holds authority, who executed, who is obliged to verify, who bears the consequence, and which artifact records that any of it occurred.

The three layers disagree. Platform controls encode real constraints: one provider prevents the developer who assigned a task from approving the resulting pull request, and withholds workflow execution until a human with write access approves it. The four tools diverge completely on merge: one withholds merge from the agent, one documents an agent that approves pull requests against configured risk thresholds, one documents merge as an action taken through its own interface, and one states only that changes should be reviewed. Attribution runs in opposite directions across providers: one records the agent as commit author and the human as co-author, another posts agent output under the human's own account. No trailer is defined for agent authorship, though one provider repurposes the co-authorship trailer for exactly that, and that provider's attribution can be disabled by configuration.

Provider terms are written in a vocabulary that predates agentic operation. Output is defined as what a service returns to the user; no term describes a service that opens a branch and authors a commit. Where the word agent does appear, in one provider's terms effective March 2026, it denotes an agent the customer builds, not the agent acting in the customer's repository. The same provider expressly obliges individual users to review, test, and validate output, and states no such duty for volume-licence customers in any document of the agreement governing them. This is a fact about drafting rather than about legal obligation, and its magnitude does not follow from the consumer-to-business transition: a second provider in the collection makes the same transition and its commercial terms retain verification vocabulary at the middle of the range, where GitHub's fall by a factor of seven and, in the instrument governing generative AI services specifically, to zero. Another provider's terms name the removal of manual review as a product feature and assign the user sole responsibility for its consequences, while that provider's documentation describes an agent that performs the approval the same document makes the user's obligation.

We report three recurring patterns: responsibility asserted with no artifact recording its discharge, execution separated from consequence with no approval event between them, and enforcement that is optional and bypassable by default. We mark which of them the framework was built to surface and which it was not. We also replace the customary three-way distinction between enforced, advisory, and absent verification with a grid separating whether a mechanism compels verification from who performs it, because one provider documents an agent occupying the approval event under thresholds a human configured once.

We argue that agentic tooling did not create this gap. A decade of code-review research already recorded that the approval artifact carries less than the terms assume, with substantive reviewer judgements going unrecorded and most changes approved in one iteration inside a day. What agentic tooling changes is that this weakness moved from a contingent property of how people work to a documented property of what a product does: a vendor now documents a product that stands at the approval event and emits the same artifact with no party capable of forming a judgement present at all.

We do not claim that any allocation is wrong, and we do not claim the gap harms anyone: showing that would require repository and incident data this study does not have. We show where accountability is asserted, where it is enforced, and where it is only assumed, and give the mapping needed to tell the three apart. The selection rule is equalised across the four tools, and every reported absence is re-tested against a doubled page set with the survival rate reported.

Keywords: accountability; agentic software engineering; AI coding agents; code review; pull request approval; terms of service; software governance; provenance; audit trail; human oversight; document analysis; responsibility allocation

ACM CCS Concepts: Software and its engineering → Collaboration in software development; Software post-development issues. Social and professional topics → Governmental regulations; Computing / technology policy. Security and privacy → Human and societal aspects of security and privacy.

Findings at a glance

#	Finding	Where
1	Verification needs two axes, not three states: whether a mechanism compels the check, and who performs it. One vendor documents an agent occupying the approval event.	§4.2, Table 2
2	The four tools diverge completely at merge: one withholds it from the agent, one delegates it to an agent, one offers it as an interface action, one says only that changes should be reviewed.	§4.2, Table 3
3	Attribution runs in opposite directions across providers, and the trailer one vendor uses for machine authorship is documented elsewhere as a multiple-human mechanism.	§4.3
4	The record dimension is populated where a vendor needs authorisation and empty where accountability would need it. At one event the platform and vendor records of the same merge disagree.	§4.3
5	Every compelling mechanism in the collection is off by default or bypassable; one vendor's merge queue requires permission to bypass the repository's merge restrictions.	§5.3, Table 4
6	Two AI-specific instruments of comparable length sit at opposite extremes of express verification vocabulary: 0.00 and 10.82 occurrences per thousand words.	§6.2, Table 5
7	Absence claims survived a doubled search six times from six, an exact binomial interval of 0.54 to 1.00.	§3.4

1 Introduction

A coding agent is assigned an issue. It creates a branch, writes code, commits it under its own name, and opens a pull request. A developer reads the diff, approves it, and merges. Something in the change later fails in production.

Two questions follow. Who was responsible for catching it, and what record exists of whether anyone tried?

Both questions have answers, and the answers are settled in two places that do not refer to each other. Platform controls determine what an agent may do and which human actions are required before it proceeds: branch protection rules, required approvals, workflow execution gates, deployment reviewers. Provider terms determine who bears the consequence of what the agent produces: ownership of output, duties to verify, allocation of liability. The first layer is written by platform engineers and enforced by software. The second is written by lawyers and enforced by contract. Neither cites the other.

This paper places them side by side.

1.1 The gap this paper occupies

Recent work has established that accountability in agent-mediated development is unsettled. Treude (2026) analysed the terms of service governing nine AI coding tools and found a consistent structure: providers grant output rights to users and allocate responsibility for correctness to them, regardless of how much autonomous action produced the output. That analysis names the problem precisely. It observes that these documents treat agent behaviour as an extension of user intent, collapsing delegation into use, and it argues that autonomy is gradual and responsibility may need to be expressed across the stages at which work is planned, proposed, executed, and verified. It then proposes a direction rather than taking it: connect responsibility to specific artifacts and events in the workflow, so that accountability is represented during normal development practice rather than reconstructed after a failure.

That direction is the one this paper follows. Where prior work read the contractual layer alone, we read it against the platform layer that operates on the same workflow, and we anchor both to the events that leave observable traces in a repository.

1.2 Research questions

Three questions organise the study, and each is answerable from published material.

RQ1. At each workflow event where an agent acts or a human gates it, what do vendor documentation and provider terms respectively state about authority, execution, verification, consequence, and record?

RQ2. Where the two layers both speak to an event, do they agree, and where they disagree, what is the shape of the disagreement?

RQ3. Which of these events produces a durable artifact from which a later question about responsibility could be answered, and which does not?

1.3 The thesis

Reading the two layers against each other returns something other than a story about agents breaking a working arrangement, and the paper argues the other thing.

An approval on a pull request records that a party holding authority performed an action the platform stores as approval. It does not record what that party understood. The gap between the act and the artifact is not new, and it is not a discovery of this paper: it was measured a decade ago and reported at the venues this paper is addressed to.

Czerwonka, Greiler, and Tilford (2015) studied Microsoft's practice under the title *Code Reviews Do Not Find Bugs* and found that "only about 15% of comments provided by reviewers indicate a possible defect, much less a blocking defect," while maintainability feedback "comprises a much larger portion of comments provided by reviewers; at least 50% of all." Their finding concerns what kind of judgement review produces rather than whether it produces one, and that is the more useful form of it here: reviewers were forming substantive judgements, and the approval artifact recorded none of their content. Sadowski et al. (2018) document a process in which "over 80% of all changes involve at most one iteration of resolving

comments” and ”70% of changes are committed less than 24 hours after they are mailed out for an initial review.” Their own characterisation is that this process is ”markedly lighter weight than in other contexts,” so we take Google as a documented light instance rather than as evidence of the norm.

What agentic tooling changes is therefore not that the artifact became thin, and not that anyone became able to notice. It is that the thinness moved from a contingent property of how people work to a documented property of what a product does. A reviewer who approved a change in twenty seconds in 2018 could have read the diff and did not. A vendor now documents a product that stands at the approval event and emits the same artifact under configured thresholds, with no party capable of reading the diff present at all. Section 4.2 gives the documentation.

The thesis of this paper is therefore that agentic tooling did not create an accountability gap in software development. It converted a long-standing weakness of the approval artifact from a matter of practice into a matter of design, while the terms in Section 4.4 continued to allocate consequence against that artifact as though nothing had changed. The Accountability Event Map locates where this has happened. The three patterns in Section 5 are what the map returns, and the two cases in Section 6 are where one provider’s own documents show the allocation and the artifact coming apart.

This framing makes a smaller claim about agents and a larger one about the arrangement they were introduced into. It does not require showing that agents caused harm, which Section 8.1 states this study cannot do, and it does not require the prior literature to have missed anything. It requires only that the approval artifact carry less than the terms assume, and that a documented path now exist by which it carries nothing at all.

Governance frameworks for agentic systems exist and allocate responsibility across organisational roles. Work in information systems has examined how agentic AI redraws the boundaries of the firm around accountability assets (Hydari and Muzaffar, 2026). Both operate above the level at which software is produced.

Work at the level of the workflow event does exist, and it is recent. Romanchuk and Bondar (2026) name a responsibility vacuum arising where authority and verification capacity do not coincide, and examine agent-generated code in continuous integration pipelines where approval steps are formally satisfied without corresponding human understanding. Nian et al. (2026) argue that no agent system can be accountable without auditability and decompose auditability into five dimensions, among them responsibility attribution and evidence integrity. Rhodes and Kang (2026) specify a runtime object binding authorisation, effect, history, and replay so that governed execution becomes attestable. These are not adjacent works. They occupy the same problem, and two of them anticipate propositions this paper reaches independently.

What none of them does is read the two layers against each other. Romanchuk and Bondar analyse organisational structure; Nian et al. and Rhodes and Kang design mechanisms that would produce the missing artifacts. All three reason about what a system should record. None asks what four specific vendors currently document, what seven specific providers currently contract for, and whether those two bodies of text agree. That is an empirical question about published material, it is answerable, and it has not been answered. Section 2.4 places this paper against that work precisely.

1.4 What we did

We collected the official documentation and governing policy documents for four widely used agentic coding tools, together with the platform controls that apply to their output and the provider terms that allocate responsibility for it. All documents were retrieved directly from their publishers, with no language model in the retrieval path, and archived in full so that every claim in this paper can be checked against the text as served.

We then mapped responsibility onto nine workflow events, from task assignment to deployment. For each

event we recorded who holds authority to act, who executed and under whose identity it was recorded, who is obliged to verify and whether a mechanism compels it, who bears the consequence, and which artifact records that the event occurred. We call the result the **Accountability Event Map**, and the five rows the **five accountability dimensions**: authority, execution, verification, consequence, and record. The map is Table 1 for one tool and Appendix A for all four.

1.5 Contributions

1. **An empirical two-layer reading, presented as the Accountability Event Map.** Prior work reasons about what an accountable agent system should record. This paper reports what four vendors currently document and seven providers currently contract for, read against each other at the events where both apply, and scored on the five accountability dimensions. The framework organising the reading is not claimed as novel; the correspondence it measures has not previously been measured.
2. **A mechanism-by-actor grid for verification**, replacing the usual three-way distinction between enforced, advisory, and absent. Whether a mechanism compels verification and who performs it are separate questions, and one provider documents an agent occupying the approval event under thresholds a human configured once. That cell is occupied and the three-way distinction has no room for it.
3. **A first measurement of how much an absence claim is worth.** Where a paper of this kind reports that a vendor documents nothing, the reader has had no way to judge the claim. Every absence reported here was re-tested against a doubled page set. Six of six survived, an exact binomial interval of 0.54 to 1.00, which is a weak estimate of a quantity nobody in this genre has previously estimated at all.
4. **Two documented cases** in which a single provider's product documentation and governing terms allocate accountability inconsistently, including one in which the removal of manual review is named as a product feature in the same document that assigns the user sole responsibility for verifying output.
5. **An archived source collection** of the documents underlying every claim, with the retrieval, extraction, absence-search, survival-test, and consistency-checking scripts, released so that the reading can be reproduced and extended as these documents change.

The paper does **not** claim to establish that the correspondence gap harms anyone. Section 8 sets out what would be required to show that, and why this study cannot.

1.6 Scope

This paper describes what the documentation and the terms say. It measures the correspondence between the two layers and the artifacts each produces. Establishing what practitioners actually do at these events, and whether the recorded artifacts match observed behaviour, requires a different study on repository data, which Section 7 identifies as the natural next step.

Before presenting the map, we position it against the three bodies of work that have approached the same question from other levels.

2 Related Work

Accountability in agent-mediated software development has been examined at three levels. Each occupies a different unit of analysis, and none works at the level of the workflow event.

2.1 The contractual layer, read alone

Treude (2026) analysed fourteen policy documents across nine AI coding assistants and agent-enabled tools, coding clauses along ownership, responsibility and liability, data governance, and delegation framing. The finding is a consistent contractual architecture: providers assign output rights to users while placing responsibility for correctness, legality, and downstream consequences on them. The paper argues that this allocation is coherent for assistive use under close supervision and becomes strained as agents plan and execute larger changes with reduced oversight, and it sets out a research roadmap whose first item is responsibility modelling in agentic workflows.

That study establishes what the contracts say. It does not compare them against the platform mechanisms that operate on the same work, and it does not ask which workflow events leave a record. Our collection overlaps that corpus and follows the delegation chains out of it, retrieving the customer agreement, the generative AI services terms, and the superseded product-specific terms that the collected documents point to, so that the terms governing enterprise agent use are on the record rather than referenced.

2.2 Governance frameworks, above the workflow

Governance frameworks for agentic systems allocate accountability across organisational roles: use case owners, risk management functions, deploying teams, and vendors. The Singapore framework is the most developed instance and treats human accountability as one of four governance pillars, proposing explicit allocation of responsibilities and adapting human-in-the-loop models to counter automation bias (Infocomm Media Development Authority, 2026).

The version read and cited here is 1.0, issued January 2026. An updated framework was published on 20 May 2026; we did not obtain it, and whether it retains the four-dimension structure described above is not established by this collection.

These frameworks are addressed to organisations deciding how to deploy agents. Their unit is the role and the use case. They do not descend to the artifacts a software team already produces, and a team that adopts one still has to decide which commit, which approval, and which merge carries which obligation.

2.3 Organisational boundaries, beside the workflow

Hydari and Muzaffar (2026) argue that agentic AI redraws organisational boundaries around accountability rather than dissolving them, introducing accountability assets as the complementary assets that make agent-supported outputs reviewable and assignable to a responsible party. They argue that verification cost and responsibility transferability determine whether execution and accountability boundaries can move together. The concern is adjacent to ours and the unit is the firm rather than the repository.

2.4 Adjacent work on review, oversight, and provenance

Several recent papers touch accountability while pursuing another question. Dorner et al. (2025) surveyed one hundred developers across five companies on how they expect code review to evolve, and identify emerging tensions concerning understanding, accountability, and trust in automation-mediated review. Kamali et al. (2026) propose a vision in which reviewers become supervisory operators of agents and treat review effectiveness as an outcome of the full review lifecycle. Velasco et al. (2026) frame provenance of generated code as a traceability and explanation problem for developers, organisations, and compliance professionals. Raees and Papangelis (2026) review reliance constructs in human-AI decision-making and find the underlying measures fragmented. Ferino et al. (2026) propose a reliance-control framework in which the level of developer control identifies overreliance and underreliance.

Each of these treats accountability as context for a different contribution. What none provides is the

mapping: which workflow event carries which obligation, under which layer, and with what artifact to show for it.

2.5 Agent accountability at the level of the event

Three 2026 papers work at or near the level this paper works at. The relationship to each is stated below from their full texts, because in two cases a reading of the abstract alone would overstate the overlap and in one it would understate it.

Romanchuk and Bondar (2026) give the condition this paper documents its mechanisms for. They define a responsibility vacuum formally: a system exhibits one for a decision if the decision occurred and no entity holds both authority over it and capacity for it. They name the resulting practice ritual review, "review in which the reviewer's action (approve/reject) is decoupled from their understanding," and observe that "the signature persists; the understanding does not." Their scope is deliberately narrow, covering decisions with irreversible or high-cost effects, and merge and deployment are their named instances.

Their method is the complement of ours, and this is the substance of the relationship rather than a difference of emphasis. They state plainly that the work "presents an organizational analysis rather than a technical audit," offer no empirical data, and use schematic examples for conceptual illustration. Their threshold parameter is explicitly uncalibrated, and they argue no calibration is needed for the claim to hold. What they supply is a formal account of a condition. What this paper supplies is the documented mechanisms through which four named products produce it: an approval event a vendor's own agent can occupy, a merge queue that requires bypassing merge restrictions, and an approval artifact indistinguishable in the record from a human's judgement on a diff. Their prediction and our observation are independent, and their coincidence is corroboration neither of us could claim alone.

Nian et al. (2026) address a different object than this paper, and the resemblance in vocabulary is misleading. They define accountability as "the ability to determine compliance and assign responsibility," auditability as "the system property that makes accountability possible," and decompose auditability into five dimensions with associated metrics. Two of those dimensions carry names close to ours. The concepts are not close. Their responsibility attribution asks whether a delegation chain can be reconstructed at runtime, measured by chain completeness and depth; our Consequence asks which party a contract makes answerable for the outcome. Their evidence integrity grades tamper-resistance on four ordinal levels from none to digitally signed; our Record asks whether an artifact exists at all. Theirs are properties of a runtime log; ours are positions in an allocation. A system could score well on all five of their dimensions while the contract governing it named no responsible party. The Cursor arrangement in Section 4.2 is close to that case: the agent's approval is attributable to a specific cloud agent identifier, minted against a published key set, so the delegation chain is recoverable and the evidence is signed. Their responsibility attribution is satisfied. The party the terms make answerable for the merged change is still the human who configured a threshold months earlier, and no dimension of theirs records that.

Their evidence is a static scan of six open-source projects, a runtime firewall evaluated on curated attacks, and controlled recovery experiments, and they state that it draws exclusively from their own tools and examines no proprietary deployment. They do not examine development workflows, pull requests, commits, or merge gates, and they analyse no vendor documentation or terms of service.

Rhodes and Kang (2026) specify an artifact adjacent to the one Section 8 argues for, and the gap between the two is instructive. Their proof of execution binds a contract, a tamper-evident causal event stream, and a replay context into an object a validator can check, issuing an attestation certificate only when five invariants hold. It is a genuine advance on recording what an agent did. It is not a record that a human verified anything, and the paper is explicit on the point: "Valid under contract does not mean safe or well-chosen contract. PoE=1 attests execution stayed within C; silent on whether C was wise." Contract authoring and review are named as upstream problems the mechanism does not address. The artifact missing from the map in Section 4 sits precisely in that upstream: not whether the agent stayed inside

its authorisation, which their certificate would establish, but whether the party bearing the consequence formed a judgement about the change.

They report overhead across three workloads: 65.9 percent on a single capability invocation, 13.6 percent on a five-node pipeline, and 4.4 percent on fifty-way parallel batches, reflecting a roughly constant 2.7-millisecond cost that amortises with concurrency. Their target setting is live agent execution under financial regulation, and they do not address code review, merge approval, or development workflows.

What the three establish, and what they leave. Between them they give a formal condition, a decomposition of what a runtime record must satisfy, and a construction for producing one. All three reason about what a system ought to record. None of the three examines what vendors currently document or what the terms governing those vendors currently allocate; each states as much. That is the space this paper occupies, and it is narrower for their existence than it would otherwise be.

2.6 Responsibility without a responsible party

The problem this paper observes at the level of the commit has a long prior life at the level of the person.

Matthias (2004) identified a responsibility gap arising where a system's behaviour is not fully predictable by its operator, so that the conditions for ascribing responsibility to a human are not met while the machine cannot bear it either. Thompson (1980) named the problem of many hands, and van de Poel, Royakkers, and Zwart (2015) recast it as a morally problematic gap in the distribution of responsibility rather than a difficulty in locating a culprit.

Elish (2019) is the closest antecedent and the most uncomfortable for the arrangement this paper documents. Analysing accidents in automated socio-technical systems, she describes a moral crumple zone: responsibility for a failure is absorbed by the nearest human operator, whose actual control was limited, in a way that protects the integrity of the technological system. That structure is the one Section 5.2 finds in the documentation: a recorded actor who is not the responsible party, an obligation attached to a human whose opportunity to discharge it is set by configuration, and consequence allocated uniformly regardless.

We do not claim to extend this literature. The five dimensions in Section 3.5 are a stipulative decomposition, chosen because each is separately observable in a repository; usefulness rather than derivation is what recommends them. The pattern is not new. What is new is that it can be read off the vendors' own published material at the level of the individual event.

2.7 Position of this paper

The contractual layer has been read. The platform layer is documented by its vendors. The requirement that agent accountability rest on recorded evidence has been argued, and mechanisms for producing that evidence have been designed. What has not been done is to read the two published layers against each other and report where they correspond, at the events that produce artifacts in a repository. That is what this paper supplies, and Section 3 sets out how the underlying sources were collected and how far the resulting claims can be pressed.

3 Method

3.1 What was collected

Two classes of document. First, the official product documentation for four agentic coding tools: GitHub Copilot cloud agent, Devin, Cursor, and Claude Code. Second, the governing policy documents that allocate responsibility for their output, comprising eighteen terms of service, customer agreements, and product-specific terms across seven providers. To these we added the platform controls that operate on agent output regardless of which tool produced it: branch protection rules and rulesets, code owners, deployment environments, commit signature verification, and the organisation audit log.

Only publisher-served documentation was collected. Blog posts, changelogs, community discussions, and third-party summaries were excluded, including where they described the same behaviour in clearer terms.

3.2 How it was retrieved

Every document was retrieved by direct request to its origin server and archived in full. No language model sat in the retrieval path at any point, so the archived text is what the server returned rather than a summary of it. Each archived file carries a header recording the requested URL, the served URL where the two differed, the HTTP status, the page title, any date string present in the served markup, and the retrieval date.

Two documents required a fallback path after returning HTTP 403 to programmatic requests, and one resolved through a permanent redirect. All three are recorded with both URLs. No document in the collection was unreachable.

Where extraction from markup could alter the text, the alteration is recorded per item. In one case a tag-stripping step removed literal angle-bracketed addresses from documents served as raw Markdown, which silently truncated a commit trailer; those documents were re-fetched without any processing and the quotations re-verified against the untouched response bodies.

3.3 What was extracted

A single rule was applied to the product documentation: extract, verbatim, every sentence stating who may perform an action, whose approval is required, or what a human must do before something proceeds.

Three rules were applied to the policy documents: extract every clause stating who must verify, review, or validate output before use; every clause allocating responsibility, liability, or indemnification for consequences; and every clause referring to autonomous, agentic, automated, or delegated operation, or to actions a product takes without a per-action human instruction.

3.4 Absence as a finding

Where a rule returned nothing, the absence was recorded together with the full list of URLs checked. Reporting the search rather than the conclusion allows a reader to judge whether the search was adequate. Two claims in this paper depend on that standard being met, so we set out how far it is met.

An absence claim was once wrong here, and the failure is instructive. A first pass over Cursor's documentation covered seven pages and concluded that the tool documented no approval step before merge. A later sweep of the same vendor's published page index found a product that approves pull requests, a merge queue, and a documented agent identity, none of which the original seven pages reached. The absence was an artifact of the search, not a property of the documentation.

The selection rule was then equalised, and the resulting coverage was not. The distinction matters and we state it before the remedy. The first collection searched five, six, nine, and sixteen pages across the four

tools, a spread of 3.2 to 1, which makes any comparison between them uninterpretable: a vendor searched three times harder will appear to document three times more. One selection rule now applies identically to all four. Candidate pages are drawn by link extraction from each vendor's own agent-documentation index, filtered to English pages whose path matches any of sixteen accountability terms and none of the exclusions, ranked by term-hit count, and capped at the same number per vendor.

The rule had to be link extraction rather than sitemap selection, because sitemaps are not uniformly available: GitHub publishes none for its documentation, and Anthropic's redirects to a platform sitemap that omits the Claude Code tree entirely. A rule depending on sitemaps could not have been applied identically, which is the defect being repaired.

The yield under the identical rule is itself a result. GitHub's documentation offers 147 pages matching the accountability terms, Cursor 23, Claude Code 10, and Devin 8. The vendors do not publish comparable quantities of material on who may act and who must check, and the first collection compounded this by searching the most prolific vendor least.

Because the yields differ, applying one rule does not produce one coverage. The final page sets are GitHub 21, Devin 14, Cursor 32, and Claude Code 19: a spread of 2.3 to 1, narrowed from 3.2 but not eliminated. Two of the four vendors simply do not publish sixteen qualifying pages. Cross-vendor comparisons in this paper are therefore bounded by a residual 2.3-to-1 difference in coverage, and a reader should treat the Devin and Claude Code columns of Appendix A as resting on less material than the others rather than as equally searched. The composition also differs in kind and not only in number: each vendor's set is the union of a purposive selection made before this objection was raised and a rule-based selection made after it.

The absences were then tested. Each absence claim was re-run against a doubled page set. Six were testable; six survived. Two could not be tested because the lexical patterns matched text in an unrelated sense on the original corpus ("write access" to a sandbox filesystem, "repository permissions" in a troubleshooting note), and reading confirmed the absence in the relevant sense, with one qualification now recorded in Appendix A.

A survival rate of six from six should be read for what it is. It says that no absence reported here dissolved when the search doubled. The exact binomial interval on six successes from six trials runs from 0.54 to 1.00, so the data are consistent with nearly half of such claims failing a wider search. It does not say that none would dissolve if the search doubled again. Two further claims could not be tested at all, and the reading that confirmed them was done by the same single coder whose reliability Section 7 reports as unestimated, so the measurement inherits the limitation it was introduced to address. It is reported because a paper that treats absence as evidence owes the reader some measure of how much an absence is worth, and no such measure has previously been offered in this genre.

One incident in producing that figure is worth recording, because it illustrates the failure mode this section is about. The first run of the survival test reported one absence refuted. On inspection the refutation was spurious: the pattern written to detect "approve PR" matched, case-insensitively and without a word boundary, the phrase "approve protected-path". Had that run been reported, the paper's headline methodological statistic would have been wrong in the direction of appearing more self-critical than the evidence warranted, an error unlikely to be challenged by a reader and correspondingly unlikely to be caught. The pattern is now anchored and the test script is deposited. Lexical instruments fail in both directions, and a result that flatters the author's caution deserves the same scrutiny as one that flatters the author's thesis.

A tool that documents no merge gate has not, on this evidence, been examined insufficiently. It has been examined under a rule applied equally to its competitors, re-examined at twice the depth, and found to document no merge gate. Lexical rules remain a way to reduce reading rather than to replace it: every absence in Appendix A was confirmed against the archived body.

3.5 The map

Nine workflow events form the columns: task assignment, plan production and approval, change authorship, pull request creation, check execution, review, approval, merge, and deployment. Events were selected on one criterion: each is described in at least one collected document as a discrete step at which an actor acts.

Five dimensions form the rows.

Authority. Who is permitted to cause this event.

Execution. Who performed it, and under which recorded identity.

Verification. Who is obliged to check, whether a mechanism compels the check before work proceeds, and which party performs it. Section 4.2 shows why the last two must be recorded separately.

Consequence. Who bears the outcome, taken from the governing terms.

Record. Which artifact shows that the event occurred.

The separation of verification from consequence is deliberate and does the analytical work in Section 5. The separation of record from the other four is what allows an obligation with no corresponding artifact to be visible as such.

3.6 Availability

The full source collection, comprising the archived documents, the extraction records, and the scripts that produced them, is deposited at <https://doi.org/10.5281/zenodo.21965182>. Documentation of this kind changes without notice, and the archive fixes the state on which every claim here rests.

3.7 Tools used in the preparation of this work

The author conceived the research question, the five-dimension framework, the nine-event decomposition, and the corpus, and wrote the first complete draft. Claude Opus 5 (Anthropic), operated through the Academic Research Skills pipeline, wrote and ran the retrieval, extraction, verification, and consistency scripts deposited with the archive; verified every reference against publisher sources; and drafted passages that the author reviewed against the archived sources. No language model is present in the retrieval or extraction path: the scripts issue direct HTTP requests, write response bodies byte-exact, and apply lexical rules, so the archive and the extraction records are reproducible from the manifest without any model. The author takes full responsibility for all of the contents of this paper, irrespective of how they were generated.

4 The Map

This section answers RQ1. Each subsection takes one of the five accountability dimensions across the nine events, and Appendix A gives the Accountability Event Map in full for all four tools. Sections 5 and 6 answer RQ2, on where the two layers disagree and what shape the disagreement takes. Section 4.3 answers RQ3, on which events leave a durable record.

4.1 Authority and execution

Authority is the most consistently documented dimension and the most consistently constrained. GitHub's documentation states that only users with write access can trigger the cloud agent, and that comments from

users without write access are never presented to it. The agent can work on one branch and open exactly one pull request per task. It cannot mark its own pull requests ready for review, and it cannot approve or merge them.

One constraint is stated with its rationale, and the rationale is the subject of this paper. GitHub prevents the user who asked the agent to create a pull request from approving it, on the ground that this maintains the expected controls in the required approvals rule. The same reasoning is applied to automations generally: work is attributed to the person who created the automation, and as when that user creates a pull request themselves, they cannot approve it.

This is an accountability boundary, encoded and enforced. It is drawn between initiating work and endorsing it, and it is drawn at the level of a specific event.

Execution divides differently. Commits are authored by the agent, with the human who assigned the task recorded as co-author. Commits are signed and appear as verified. Each commit message carries a link to the agent session logs.

Across providers this arrangement does not hold. Devin’s documentation states that commits made by the chat agent appear as the Devin bot, while comments a user writes through the review interface appear under that user’s own GitHub identity, and that suggested changes applied by a user are authored by that user in the ordinary way. Claude Code’s web documentation states that replies posted to review threads use the human’s GitHub account and appear under the human’s username, distinguished only by a label in the body of the reply.

The direction of attribution is therefore inverted between providers. In one arrangement the agent is the recorded actor and the human is secondary. In another the human is the recorded actor and the agent is a label. Both are current, documented, and applied to the same artifact type in the same platform.

Table 1. The accountability map, GitHub Copilot cloud agent.

Event	Authority	Execution and recorded identity	Verification	Consequence	Record
Task assignment	Write access only	Human	n/a	User	Issue assignment
Plan approval	Not documented	Not documented	Absent	User	None
Change authorship	Agent, single branch	Agent as author, human as co-author	n/a	User	Signed commit, session log link
Pull request creation	Agent, draft only	Agent	n/a	User	Pull request
Check execution	Write access	Human, approve and run	Enforced by default, disableable	User	Workflow run
Review	Any reviewer	Human	Advisory	User	Comment, if submitted
Approval	Not the assigning user	A different human	Enforced where configured	User	Review submission event
Merge	Human only	Agent cannot merge	Structurally enforced	User	Merge event
Deployment	Named reviewers	Human	Optional	User	Environment approval

The consequence column is constant across every row. The verification column varies on two axes, separated in Table 2, and Appendix A shows that its agent-performed cell appears only when the map is extended to all four tools. The record column contains one empty cell and one conditional cell. That combination is what Section 5 examines.

4.2 Verification

Verification is usually described in three states: a mechanism withholds progress, or the documentation advises, or nothing is said. That description conflates two independent questions, and the conflation only becomes visible when a vendor occupies a cell the three-state scheme has no room for.

The two questions are: does a mechanism compel verification before the work proceeds, and who performs it. A required approval and an agent-performed approval are both compelling; they differ in who acts. An advisory sentence and a silent page both compel nothing; they differ in whether an obligation was stated. Crossing the two gives the grid below, and the map is easier to read once the axes are separated.

Table 2. Verification by mechanism strength and acting party.

	Performed by a human	Performed by an agent
Compelled: progress is withheld until it happens	Required approvals; approval by non-pusher; workflow execution approval on agent pull requests; plan approval before edits	Cursor PR Routing and Approval, below a configured risk threshold; merge queue admission
Not compelled: nothing withholds progress	Advisory instruction to review before merging	Bugbot and Security Agents reviewing every pull request; Claude Code auto-fix responding to CI failures and reviewer comments; Devin Review posting commit status checks

The cell that the three-state scheme cannot express is the top right. It is occupied, by one vendor, and the occupancy is the substantive finding of this section.

The bottom right is occupied too, and by three vendors rather than one, which is worth noting because it is where the volume is. Agents that review without gating produce findings, comments, and status checks at every pull request. They generate most of the verification-shaped output a team now sees, and none of it withholds anything. A team reading its own pull request page encounters agent-generated review activity in the same visual register as a human's, and the distinction between the cell that compels and the cell that does not is not marked there either.

Compelled, human-performed. A mechanism withholds progress until a human acts. Workflow execution on agent-authored pull requests is the clearest instance: GitHub states that workflows are not triggered until the agent's code is reviewed and a user with write access clicks to approve them. Required approvals, when configured, compel in the same sense.

Not compelled, obligation stated. The documentation instructs but nothing withholds. GitHub tells reviewers to check the pull request thoroughly before merging, and states that the agent should be used as a tool rather than a replacement, with content reviewed and tested before merging. Cursor's guidance states that standards for what gets merged should be the same whether code was written by hand or by an agent. Claude Code's action documentation says to review changes before merging.

Compelled, agent-performed. Cursor documents a product that occupies the approval event: PR Routing & Approval "can approve low-risk pull requests when your approval criteria are met," subject to a "Maximum Risk Threshold [that] sets the highest risk level the agent may approve." It withholds approval where an AI reviewer has reported findings needing human review, and its own documentation states that it "does not replace a full code review." Configured through the same vendor's automations, an agent granted approval rights "can also approve, request changes, and dismiss reviews."

The threshold genuinely compels: a pull request above it is not approved. What has changed is not whether a gate exists but who stands at it. The artifact produced is an approval, and nothing in the record distinguishes it from one a human produced by reading the diff. An agent that can also dismiss reviews can retract a human's judgement without the record showing that a judgement was retracted by a non-human party.

Not compelled, nothing stated. Across nineteen Claude Code documentation pages under the equal-effort rule, no page requires that a human approve or confirm before an agent’s changes are merged, and the tool withholds no merge capability pending human action. The advisory sentence quoted above is the whole of the obligation. Verification at the tool-call level is compelled (“Claude Code requires approval before running Bash commands that can modify your system”), but that gate sits inside the session, not at the merge.

The distinction between the cells is not marked by the documents themselves. A team reading vendor documentation encounters compelling gates, advisory sentences, and a product that approves pull requests, all in the same prose register and with nothing to signal which is which.

Table 3. Cross-provider divergence.

Provider	Documented merge gate	Attribution direction	Verification duty in terms
GitHub Copilot	Enforced: agent cannot approve or merge its own pull request	Agent author, human co-author	Individual: explicit. Volume licence: none
Devin	Advisory: human review recommended before merging; merge and auto-merge available as actions in the vendor’s interface	Bot for agent commits, user identity for user comments	n/a
Cursor	Compelled, agent-performed: agent approves below a configured risk threshold; merge queue requires bypass of merge restrictions	Not documented for commits; OIDC workload identity documented	Explicit, in clause 1.4, alongside clause 1.7
Claude Code	Advisory only: review changes before merging	Human’s own account for review replies; agent named in a co-authorship trailer on commits, disableable	n/a

Table 4. Verification status by mechanism.

Mechanism	Compels?	Acting party	Default
Workflow execution approval	Yes	Human	On, disableable
Required approvals	Yes	Human	Off
Approval by non-pusher	Yes	Human	Off
Stale approval dismissal	Yes	Human	Off
Code owner review	Yes	Human	Off
Deployment reviewers	Yes	Human	Off, admin bypass on
Plan approval before edits, Claude Code and Cursor	Yes	Human	On in plan mode; plan mode is opt-in
Permission prompt before system-modifying commands	Yes	Human	On, removable in autonomous modes
Agent approval below risk threshold, Cursor	Yes	Agent	Off
Merge queue admission, Cursor	Yes	Agent	Off; requires bypass of merge restrictions
Merge through the vendor interface, Devin	No	Human	n/a; human review advised, not withheld
Merge gate, Claude Code	No	n/a	n/a
Reviewer instruction in documentation	No	Human	n/a

Every compelling mechanism in this collection is off by default except two, and both of those are removable. The two mechanisms whose acting party is an agent are also off by default, which means the arrangement a team gets without configuring anything is the one in which nothing is compelled at the merge.

4.3 Record

The record dimension is where the map is thinnest, and the thinness is the finding.

Some events produce durable artifacts. A commit exists. A pull request exists. A review submission is recorded in the organisation audit log, as is a merge.

Others do not. The audit log's pull request category contains no event corresponding to an approving review specifically; approval is recorded as the submission of a review, which is the same event type as a comment or a request for changes. The distinction between endorsing a change and remarking on it is not carried by the record.

The audit log's field lists include markers for whether an actor is an agent, whether an actor is a bot, and an agent session identifier. The page defines none of them. The fields exist and their meaning is not documented.

No trailer is defined for agent authorship in any standardised form. GitHub documents two attribution trailers, one for co-authorship and one for acting on behalf of an organisation. Neither mentions AI, agents, or machine authorship; the first is documented purely as a multiple-human-author mechanism. The Git project's own trailer documentation defines the mechanism and offers a sign-off trailer as its example, defining no attribution trailer for agents and maintaining no registry of trailer keys.

The absence of a definition has not prevented use. Claude Code's settings documentation records that its default commit attribution is a co-authorship trailer naming the model, and that the model name in the trailer reflects the model active for that session. The trailer documented as a multiple-human-author mechanism is, in practice, the mechanism by which one major vendor records machine authorship and model provenance. There is no registry to record that this is what the key now means, and no definition against which another vendor could implement it the same way.

Where attribution is produced, it can be removed. Claude Code's settings documentation states that setting the commit and pull request attribution values to empty strings hides attribution entirely.

This is the answer to RQ3, and it is a short one: of the nine events, commit authorship, pull request creation, check execution, and merge produce durable artifacts; review and approval produce one only conditionally and do not distinguish endorsement from comment; plan approval and, at three of four tools, deployment produce none; and the verification the terms require produces none anywhere.

The record dimension is not empty everywhere, and where it is populated is informative. Cursor documents a cryptographic identity for its cloud agents: a token minted inside the agent's own environment, carrying claims that include a stable owner subject, a service account identifier, an owning team, and a cloud agent identifier, verifiable by any relying party against a published key set. This is a durable, checkable record that a particular agent acted.

It attaches to the agent's outbound requests to other systems, not to the commit it authors. The artifact exists where the vendor needs it to authorise the agent against third-party infrastructure, and does not exist where a reader of the repository would need it to answer who produced a change. The record dimension is thin at the accountability events not because such artifacts are difficult to produce, but because they have been produced for a different purpose.

At one event the record is not merely thin but inconsistent between the layers that write it. Cursor's merge queue documentation notes that after a successful run, the original pull request "may appear as closed in GitHub instead of merged, even though [the product] treats it as merged across the product." Two systems record the same event, and they disagree about whether it happened. A later question about when a change entered the default branch, and on whose authority, resolves differently depending on which layer is asked.

The verification event, which the terms in Section 4.4 make the user's obligation, produces no artifact of its own anywhere in the map. Where an approval artifact does exist, Section 4.2 has shown that it no longer reliably indicates that a human read the diff.

4.4 Consequence

The governing terms allocate consequence uniformly and without reference to the distinctions above.

Users are made responsible for evaluating output and for what follows from using it. Section J of GitHub's terms of service, effective April 2026, states that the user is responsible for reviewing, testing, and validating any output before use, and that the user is responsible for its use, with indemnity obligations extending to claims arising from output incorporated into products. Cursor's terms state that the user is responsible for evaluating and bearing all risks associated with the use of any suggestion. Sourcegraph's terms state that the user is solely responsible for reviewing and validating outputs before use. JetBrains states the duty within an enumerated list of user responsibilities.

Two documents state no verification duty at all. Replit's terms disclaim responsibility for accuracy without assigning a duty to check. The GitHub terms document governing additional products allocates neither verification nor responsibility for output, delegating both.

None of these documents allocates responsibility per event. The duty attaches to output as a class, and the terms in which output is defined are examined in Section 6.

5 Three Patterns

A framework built with a Record dimension will find empty Record cells, and a reader is entitled to ask whether the patterns below are discoveries or restatements of the decision to look. The question applies unevenly across the three, and we mark where it bites.

The first two patterns are, in part, what the decomposition was built to surface. Separating Record from the other four dimensions makes an obligation with no corresponding artifact visible, and Section 5.1 duly reports one; separating Execution from Consequence makes their divergence visible, and Section 5.2 duly reports that. What is not guaranteed by the framework is the *direction* and *extent*: the decomposition could have shown a record at every event, or execution and consequence coinciding throughout, and it does not. The contingent part is the finding; the visibility is the instrument.

The third pattern carries no such qualification. That branch protection ships off by default, that rulesets carry bypass lists, that deployment environments permit administrator override, and that one vendor's merge queue requires permission to bypass merge restrictions are facts about how vendors ship defaults. No framework predicts them. They could have come out the other way, and at some vendors on some settings they do.

5.1 Responsibility without record

The terms make verification the user's obligation. The platform records no event when it is performed.

A user who reviews an agent's pull request, reads the diff, and forms a judgement produces an artifact only if they also submit a review. If they approve, the approval is recorded as a review submission, indistinguishable in the audit log from a comment. If they inspect and merge without submitting a review, which the platform permits unless required approvals are configured, no artifact records that inspection happened.

The obligation is therefore asserted in a layer that cannot observe whether it is discharged, and discharged in a layer that does not record it as discharge. Nothing in the collected documentation connects the two.

This is not an argument that verification is not happening. It is the observation that whether it happened is not recoverable from the artifacts the workflow produces, at the one event where the terms place the duty.

The observation is old, and Section 1.3 gives its history: reviewers at Microsoft were recorded producing substantive maintainability judgements that the approval artifact did not carry, and reviewers at Google were recorded approving in a single iteration inside a day. In neither case did the artifact record what was understood. What the documentation collected here adds is a case the earlier studies could not have found, because it did not exist. In every arrangement those studies observed, a party capable of forming the judgement stood at the event and the artifact simply failed to record what they formed. Table 2's agent-occupied cell is the first documented arrangement in which no such party is present. The artifact is unchanged; what it can be evidence of is not.

5.2 Execution separated from consequence

In the arrangement the terms assume, the party who acts is the party who bears the outcome. Across the collected documentation, the two separate at identifiable events.

Commit authorship is the first. The agent is recorded as author and the human as co-author, while the terms place responsibility on the human. The recorded actor and the responsible party are different parties by design, and the design is stated as a traceability feature.

This is the structure Elish (2019) describes. Responsibility settles on the nearest human to the failure, whose actual control over the outcome was bounded by what the configuration allowed, and the arrangement holds the technological system harmless. What the documentation adds to her account is the mechanism: the boundedness is not a matter of attention or skill but of defaults, and Section 5.3 shows the defaults are off.

Automatic merging is the second. A pull request configured for auto-merge merges when required reviews and status checks are satisfied, which may be after the approving human has moved on. The mechanism is disabled if someone without write permissions pushes new changes, which leaves changes pushed by parties that do have write access, including agents operating on the branch, outside the disabling condition. Two optional controls exist to close this: dismissing stale approvals when new commits affect the diff, and requiring that the most recent reviewable push be approved by someone other than the person who pushed it. Both are off unless enabled.

The third case is the most direct, and Section 6 treats it separately: a provider whose terms describe a feature that executes code without manual review or confirmation, and assign the user sole responsibility for the result.

5.3 Optional enforcement

Every enforcement mechanism located in this collection is optional, bypassable, or both.

Required approvals apply only where configured. Branch protection restrictions do not apply to users with admin permissions or the bypass permission by default, and applying them to administrators is an opt-in. Rulesets carry explicit bypass lists. The gate on workflow execution for agent pull requests is on by default and can be turned off by a repository setting. Deployment environments allow administrators to bypass protection rules and force deployments by default. Attribution can be set to empty.

One instance is worth stating precisely, because the direction of the accommodation is informative. GitHub's documentation notes that a rule permitting only specific commit authors can prevent the cloud agent from creating or updating pull requests, and that access to the agent will be blocked where an incompatible rule is configured. The remedy it offers is to add the agent as a bypass actor on the ruleset.

A second vendor inverts the relationship more directly still. Cursor's merge queue routes pull requests into a queue that "decides what runs next, what needs CI, and what can safely merge," and its stated prerequisites include that the repository be configured "to let the App push or bypass the right merge

restrictions.” The control that would enforce the obligation is not merely optional here; the vendor-supplied mechanism that replaces it requires permission to bypass it.

The pattern across these is consistent. The controls that would enforce the obligations the terms assert are present, and they are configuration rather than default. Two providers show the pattern in concentrated form, and we treat them individually.

6 Two Documented Cases

6.1 Requiring verification and selling its removal

Cursor’s terms of service, last updated 13 August 2026, contain both of the following.

On verification, in clause 1.4: the user agrees that they are responsible for evaluating, and bearing all risks associated with, the use of any suggestions, including any reliance on their accuracy, completeness, or usefulness.

On a product feature, in clause 1.7: the service may include a feature that automatically executes code suggestions without manual review or confirmation, “and will be clearly labeled accordingly.” By enabling it the user acknowledges assuming all risks associated with the execution of automatically generated code, including system outages, software defects, data loss, and security vulnerabilities, and is solely responsible for any impact resulting from its use, including ensuring appropriate safeguards, testing, and monitoring are in place.

The second clause names the removal of the review step described by the first as a product feature, and reallocates nothing. The duty to evaluate is unchanged; only the opportunity to perform it is removed. The labelling requirement discloses that removal to the user who enables it. It does not restore the step, and it does not move the duty.

The pattern is not confined to one provider. GitHub’s responsible-use documentation defines a permission prompt as “an interactive confirmation step . . . that asks the user to approve an action — such as modifying a file, executing a command, or accessing files outside the current directory — before the agent proceeds,” and calls permission prompts “a key safety mechanism for local agentic execution.” The same document describes autopilot mode: “you will grant it full permissions to allow it to complete a task autonomously, without requiring you to approve activity as it works on the task.” A mechanism named a key safety mechanism, and a mode that removes it, in one document, from the provider whose terms of service carry the explicit duty to review, test, and validate.

Against this, Cursor’s documentation. Thirty-two pages were checked, covering agent modes and overview, review guidance, permissions, security and run modes, cloud agents and their capabilities, identity, automations, the GitHub integration, the review product, the pull request page, and the merge queue, together with the rule-selected set of Section 3.4. No page states that a human must approve, review, or confirm before an agent’s changes are merged. What the pages do describe is a product that performs the approval itself, below a risk threshold the user configures, and a merge queue whose prerequisites include permitting the vendor’s app to bypass the repository’s merge restrictions.

What stands together in this provider’s own material is therefore sharper than an unenforced duty. The contract requires the user to evaluate suggestions and bear all risk of relying on them. One product feature removes the opportunity to evaluate and assigns the consequences to the user. Another performs the evaluation on the user’s behalf, through an agent applying criteria the user configured once, and records its judgement as an approval. The duty never moves. What moves is who discharges it, and at no point is the party bearing the consequence required to perform the act the contract asks of them.

6.2 A product that describes an agent and terms that describe a return value

GitHub's product documentation describes an autonomous agent in explicit terms. It has access to the code and can push changes. It authors commits. It opens pull requests. It runs in its own ephemeral environment. It is prevented from approving its own work because approval is a control that must be preserved.

GitHub's terms describe something else.

Section J of the terms of service, effective April 2026, is the section that governs AI features. It defines an AI feature as one that uses machine learning or artificial intelligence to generate output, input as content provided to such a feature, and output as responses and suggestions generated by it. It states the verification duty quoted in Section 4.4.

Section J contains no instance of agent, agentic, autonomous, automated, delegated, on your behalf, without human, unattended, or take action. The vocabulary is input, output, and feature throughout. The relationship it describes is one in which a user supplies content and receives content back.

The only provision in the entire terms of service that addresses acting through a non-human party is in the account requirements. It permits machine accounts, describes a machine account as one set up by an individual human who accepts the terms on its behalf and is responsible for its actions, notes that multiple users may direct those actions, and states that the account owner is ultimately responsible for the machine's actions. This is a provision about automated task accounts, and it is the contractual construct under which agentic work currently sits.

Section J is also the only lettered section of the document without an internal plain-language summary, a feature every other section carries.

The terms governing enterprise customers were replaced in March 2026, and the replacement is instructive on exactly this point. It introduces the word agent, four times, in a single sense: the customer is solely responsible for any application or agent they create using or for use with the service, and for third-party agent templates they install. The agent contemplated is one the customer builds. The agent that opens pull requests in the customer's repository is still described only through its output, which the definitions state is what the service returns to the user.

That document contains no duty to verify, review, validate, or test, and neither does the agreement it forms part of. It states that it combines with the customer's volume licensing agreement, the GitHub Customer Agreement's General Terms, to form the Agreement. We searched all three documents, together with the Enterprise Cloud product-specific terms, for twenty-one terms expressing a duty to check output before use. In the Generative AI Services Terms, all twenty-one return nothing. In the General Terms and in the Enterprise Cloud terms, one term appears, in neither case imposing a duty to check output. In the terms of service governing individual users, nine of the twenty-one appear. The full search, term by term with hit counts, is recorded in the archive.

The single clause in the enterprise document that allocates responsibility is headed Shared Responsibility. It reads: "While GitHub is responsible for our commitments in this document, you are solely responsible for any application or agent you create using (or for use with) Generative AI Services." We note the heading without drawing a rhetorical inference from it: shared responsibility is settled vocabulary in cloud contracting, denoting a division between provider-secured infrastructure and customer-secured configuration, and it has never connoted symmetry. What the clause does is locate the customer's responsibility in agents the customer builds. The agent that opens a pull request in the customer's repository is not an agent the customer built.

The result is a division in what the instruments expressly allocate. An individual user, governed by the terms of service, carries an explicit contractual duty to review, test, and validate output before use. A volume licence customer, governed by the enterprise agreement and the party most likely to operate agents at scale, is subject to no such express duty in any document of that agreement.

Two qualifications are necessary, and the second is a finding.

Absence of a term is not absence of a duty. Contracts are not exhaustive statements of obligation. Duties arise from background law whether or not an instrument recites them: from negligence, from implied terms as to reasonable skill and care, and from statute. A customer whose agreement says nothing about verifying output is not thereby relieved of any obligation to verify; the question is left to the general law. What the search establishes is a fact about drafting (which duties the parties chose to write down) and not a fact about what either party owes. Every claim in this section should be read in that narrower sense, and the abstract has been amended to match.

The obvious explanation does not survive the corpus. A reader with contract-drafting experience will observe that consumer-facing terms routinely carry express duties and warnings because consumer protection regimes require salient disclosure to unsophisticated parties, while negotiated business instruments omit them because the parties are sophisticated, background commercial law fills the gap, and express duties on a customer are a term customers resist. On that account the asymmetry is a drafting convention with no accountability significance at all.

The collection permits this to be tested, because it contains another provider with both a consumer and a commercial instrument. Applying the same twenty-one-term search:

Table 5. Express verification-duty vocabulary across the collection. Distinct terms is a coverage measure and is confounded by length, since a longer document has more opportunity to contain any given term at least once; occurrences per thousand words controls for that. Both are reported because they do not say the same thing.

Instrument	Words	Distinct terms, of 21	Occurrences	Per 1,000 words
GitHub Terms of Service (individual)	7,457	9	13	1.74
GitHub Customer Agreement, General Terms (volume)	3,956	1	1	0.25
GitHub Generative AI Services Terms (volume)	951	0	0	0.00
GitHub Enterprise Cloud product terms (volume)	1,041	1	1	0.96
Anthropic Consumer Terms	4,703	6	11	2.34
Anthropic Commercial Terms	3,794	6	6	1.58
JetBrains AI Service terms	6,694	6	8	1.20
Replit terms	3,630	2	4	1.10
Cognition terms	1,534	1	1	0.65
Sourcegraph Terms of Service	6,908	2	4	0.58
Sourcegraph AI Terms of Use	647	5	7	10.82
Sourcegraph Cody notice	981	1	3	3.06

The two measures disagree, and the disagreement is the result.

On coverage, Anthropic's consumer and commercial instruments are identical: six of twenty-one in each. On density they are not: 2.34 per thousand words against 1.58. The consumer-to-commercial transition is therefore associated with some reduction in express verification vocabulary at both providers, and the drafting-convention account has real force. It is not the case, as coverage alone would suggest, that one provider is unaffected by the transition.

What the account does not explain is the magnitude. Anthropic's reduction is a factor of about 1.5, landing its commercial terms at 1.58, the middle of this collection, beside JetBrains at 1.20 and Replit at 1.10. GitHub's is a factor of seven, to 0.25 in the General Terms, and to zero in the instrument that governs generative AI services specifically. A convention that predicts thinning does not predict thinning to nothing in the one document addressed to the product at issue.

A sharper comparison is available within the table, and it does not depend on the consumer-to-business

boundary at all. Two providers in this collection publish an instrument addressed specifically to AI services. GitHub's, the Generative AI Services Terms, runs to 951 words and contains none of the twenty-one terms. Sourcegraph's AI Terms of Use runs to 647 words and contains five of them at 10.82 occurrences per thousand words, the highest density in the collection, including the sentence "You are solely responsible for reviewing and validating any Outputs before use." Two documents of comparable length, both written for the same product class, both addressed to business customers, at opposite ends of the range. Whatever explains that, document length and the sophistication of the counterparty do not.

We do not claim the reverse, that GitHub's drafting is deficient. Term counting is a crude instrument, the documents serve different commercial structures, the shortest instruments produce unstable densities, the comparison rests on two providers rather than a sample, and background law may well supply for GitHub's customers what Sourcegraph and Anthropic state expressly. The claim is only that the size of the gap is not accounted for by the boundaries it straddles, and that it therefore remains worth reporting.

One row in an earlier version of this table was wrong, and the correction is recorded because it bears on how the table should be read. Sourcegraph's terms were first archived from sourcegraph.com/terms, which is a 266-word index page listing the instruments rather than an instrument itself. It was reported at zero, which read as a provider imposing no express duty. Retrieving the documents the index points to reversed the finding entirely. A short capture that yields no matching vocabulary and a genuine absence of vocabulary are indistinguishable in a table of counts, and only reading the document tells them apart.

One further observation on the same chain. The terms document effective April 2026 directs Copilot Business and Copilot Enterprise users to a product-specific terms document that is published under the title "GitHub Copilot [Archive]" and states, in its first line, "These terms have been deprecated effective 5 March 2026." A current document points at a superseded one.

7 Threats to Validity

Documentation is not practice. This paper reads what providers publish. What teams do at these events, and whether the artifacts produced match observed behaviour, requires repository data.

One retrieval date. These documents change without notice. Every claim is fixed to the archived text, which is deposited for that reason, and any of it may have moved since.

Four tools and eighteen policy documents. The tools were selected for prominence and the documents for their governing relationship to those tools. A different selection could produce a different distribution of the patterns in Section 5.

Community conventions were not collected. Section 4.3 reports the absence of a standardised agent attribution trailer in official documentation. Whether practitioners have converged on conventions of their own is a separate question this collection does not address.

Referenced documents remain outside the collection. Several documents named in the collected terms were not retrieved, including the Microsoft product terms applicable to purchases made through Microsoft, to which the Generative AI Services Terms explicitly route customers who buy through that channel, and the acceptable use policies. Each is named in the archive with its referring sentence. Three GitHub product documentation pages returned HTTP 404 at retrieval and are recorded as unreachable rather than treated as absences.

A single coder, with no reliability estimate. Every classification here (in particular the assignment of a mechanism to a cell of Table 2, which is the central analytic act) was made once, by one party, with no independent second coding and no agreement statistic. A reader has no measure of how much of the classification is reproducible and how much is one reading. This is the most serious limitation of the study.

It is stated rather than remedied because a second coder was not available; the archive, the rules, and the scripts are deposited so that anyone may apply one and test the classification.

This limitation interacts with the remedy applied to the previous one, and the interaction runs the wrong way. Equalising search effort raised the collection from 66 archived documents to 116. Every additional classification was made by the same single coder, so the volume of unverified coding grew by roughly three quarters at the same time as the sampling improved. The study is better designed than it was and carries more unaudited judgement than it did. Both statements are true and the second should not be lost behind the first.

Documentation moves under its own citations, twice in one study period. Between the first retrieval underlying this work and its revision, GitHub renamed Copilot coding agent to Copilot cloud agent, and Anthropic moved the Claude Code documentation from `docs.claude.com/en/docs/claude-code/` to `code.claude.com/docs/en/`. Paths in the original collection containing coding-agent now return 404 or redirect, and the Claude Code paths redirect to a different domain. Half the vendors studied restructured their documentation within a few months. The archive records both requested and served URLs for every item, which is the only reason either change is visible rather than silently absorbed, and it is a reason to treat any claim in this literature as dated on publication.

Absences are absences of documentation, and of a particular search. A tool that documents no merge gate may enforce one. The finding is about what a team reading the vendor's material can determine. Section 3.4 records a case in this work where a reported absence was overturned by a wider sweep, reports the equal-effort rule adopted in response, and gives the survival rate of the remaining absence claims under a doubled search. Six of six survived. With six testable claims that figure carries wide uncertainty, and the possibility that a wider sweep still would overturn another is not excluded.

Term counting is not contract interpretation. Section 6.2 counts the presence of verification vocabulary across instruments. That measures what parties chose to write down. It does not measure what they owe, which depends on background law the instruments do not recite, and no conclusion about legal obligation should be drawn from Table 5.

8 Implications

The recommendations below are conditional, and Section 8.1 states the condition. This study establishes that the two layers do not correspond at the events where agentic work happens. It does not establish that the lack of correspondence costs anything. Each recommendation therefore reads: if a team, platform, or regulator wants these events to be answerable after the fact, here is what currently prevents it. Whether they should want that is not settled here.

For platforms. The events that carry obligations should produce artifacts that distinguish them. An approving review is a different act from a comment, and an audit log that records both as a review submission cannot support a later question about who endorsed a change. Fields already exist to mark an agent actor; documenting what they mean would let the same log answer whether a change was authored by an agent, by whom it was approved, and whether the two were the same party.

The artifact this calls for is not the one the runtime-attestation literature is building. Rhodes and Kang (2026) can certify that an agent stayed inside an authorised scope, and say explicitly that such a certificate is silent on whether the scope was well chosen. The missing record sits upstream of theirs: not that the agent acted within its authorisation, but that the party the terms make answerable formed a judgement about the change before it merged. A platform that produced both would answer two different questions, and only the second is the one Section 4.4 makes the user's obligation.

For providers. Terms written in the vocabulary of input and output describe a relationship the same provider's documentation no longer describes. A definition of output as what a service returns to a user does not reach a service that creates a branch, authors a commit, and opens a pull request. Naming the acting party is a precondition for allocating responsibility for the act.

For teams and for compliance work. Whether a mechanism compels verification, and who performs it, is not visible in vendor prose and determines whether a stated obligation is met by configuration, by an agent, or by habit. The map in Section 4 identifies which events currently produce an artifact and which do not.

8.1 What this paper does not establish

This study reports a correspondence gap between two published layers. It does not establish that the gap harms anyone, and the distinction matters enough to state rather than leave to inference.

We have not shown a dispute that turned on a missing artifact, a post-incident review that could not reconstruct who approved a change, a regulatory finding, or a team impaired by the arrangement. The scenario in Section 1, a change that fails in production and a question about who was responsible for catching it, is a motivating illustration and is not evidence. Nor do we cite a compliance regime that presently requires artifacts at these events; the claim that such regimes are coming is a forecast, and we withdraw it as a premise.

There is a coherent position on which the gap is not a defect at all. Contracts are drafted abstractly because products change faster than instruments can; documentation is concrete because engineers must act. A duty framed against a class of output rather than a list of events survives a product redesign, and a contract enumerating obligations at the granularity of Table 1 would be obsolete on signature. On that view the two layers are correctly specialised and their failure to refer to each other is a design, not an oversight. A related argument holds that verification is not usefully recordable: a human reading a diff produces no natural trace, and mandating one produces an artifact of compliance rather than of judgement.

We do not refute either. Section 1.1 sets out the reading this paper does argue, and Section 9 returns to it. What the map establishes is narrower than the thesis and independent of it: at the events where agentic software work happens, an approval artifact does not indicate that a person read the change, and nothing in either layer marks where that is so. Whether that condition costs anything is a question for a study of repositories and incidents, which Section 7 identifies as the natural successor to this one.

9 Conclusion

A coding agent authors a commit, and a human approves the pull request that carries it. Both acts leave traces. Which of them constitutes the verification the governing terms require is not answerable from those traces, because the terms attach the duty to a class of output while the platform records events, and the two layers are drawn up independently of each other.

The Accountability Event Map presented here covers nine workflow events across four tools and the governing documents that allocate responsibility for their output, scored on five accountability dimensions, and it returns three recurring patterns: obligations asserted with no artifact recording their discharge, execution separated from consequence with no approval event between, and enforcement that exists as configuration rather than default. Two providers illustrate the divergence directly. One requires users to evaluate suggestions in the same document that sells a feature removing the opportunity to do so, and documents a second product that performs the evaluation on the user's behalf. Another describes an autonomous agent in its documentation and, in terms effective four months later, a service that returns output to a user.

The map also returned an arrangement the customary vocabulary cannot express. Verification is described as enforced, advised, or absent, but those three collapse two independent questions: whether a mechanism compels the check, and who performs it. At one provider the mechanism compels and the performer is an agent, applying criteria a human set once and producing an approval that the artifact cannot distinguish from a human's judgement on a diff. An approval event that no longer implies a person read the change is the clearest instance of the correspondence gap this paper set out to measure. Whether it costs anything is the question Section 8.1 declines, and the question a study of repositories and incidents would have to answer.

None of these allocations is wrong on its own terms. Each layer is internally coherent. What the map shows is that they were built to answer the same question and have not been read against each other, and that the events where they disagree are the events at which agentic software work actually happens.

It would be convenient to conclude that agents broke this, and the evidence does not support it. The approval artifact recorded less than the terms assume long before agents existed, and the software engineering literature measured that a decade ago: reviewers produced substantive judgements the artifact never carried, and changes were approved in a single iteration inside a day. What the documentation collected here establishes is narrower and harder to set aside. In every arrangement those studies observed, a party who could have formed a judgement stood at the event. A vendor now ships a product for which that is not so, and the terms allocate consequence against the resulting artifact regardless. The weakness did not appear. It stopped being a matter of how carefully people work and became a matter of what a product is built to do.

Data Availability

The complete source collection underlying this paper is deposited at Zenodo under <https://doi.org/10.5281/zenodo.21965182>. It comprises 121 manifest items retrieved on 16 August 2026, of which 118 returned HTTP 200 and were archived, and 3 returned HTTP 404 and are recorded as unreachable with the URLs attempted. Each item carries a header recording the requested URL, the served URL where these differed, the HTTP status, the page title parsed from the served markup, any date string present in that markup, the SHA-256 of the body as served, the byte length, and the retrieval timestamp and method. Bodies are stored byte-exact alongside the headered copies, so any extraction can be re-checked against the untouched response. Twenty-three items were served from a URL other than the one requested; both are recorded. No item required a fallback retrieval path at this retrieval.

Six extraction records accompany the archive, one per document group, containing the passages returned by the extraction rules together with the rules themselves. A separate absence record lists, per policy document, every term searched and its hit count, so that a claim of absence can be judged against the search that produced it. Items whose served body yielded too little text to support an absence claim are marked as such rather than counted as absences.

No language model was used at any point in the retrieval or extraction path; both are performed by scripts included in the deposit, and the manifest, the retrieval script, the extraction script, the absence-search script, and the consistency checker are deposited with the archive so that the collection can be rebuilt, its coverage extended, and the counts in this paper re-checked against it. Where markup processing could alter served text, the alteration is recorded per item and the affected documents were re-retrieved without processing. Section 3.7 records which parts of this paper were produced with AI assistance, including the authorship of those scripts.

References

- Czerwonka, J., Greiler, M., and Tilford, J. (2015). Code Reviews Do Not Find Bugs: How the Current Code Review Best Practice Slows Us Down. In *Proceedings of the 37th International Conference on Software Engineering (ICSE '15)*, Vol. 2 (pp. 27–28). IEEE Press.
- Dorner, M., Bauer, A., Šmite, D., Thode, L., Mendez, D., Britto, R., Lukasczyk, S., Zabardast, E., and Kormann, M. (2025). Quo Vadis, Code Review? Exploring the Future of Code Review. arXiv:2508.06879 [cs.SE].
- Elish, M. C. (2019). Moral Crumple Zones: Cautionary Tales in Human-Robot Interaction. *Engaging Science, Technology, and Society*, 5, 40–60.
- Ferino, S., Hoda, R., Grundy, J., and Treude, C. (2026). Towards an Appropriate Level of Reliance on AI: A Preliminary Reliance-Control Framework for AI in Software Engineering. In FSE Companion '26, 2nd Workshop on Human-Centered AI for Software Engineering, Montreal, 5 to 9 July 2026. ACM. arXiv:2604.10530 [cs.SE].
- Hydari, M. Z., and Muzaffar, F. (2026). Redrawing the AI Map: A Theory of Accountability Boundaries in Agentic Ecosystems. arXiv:2605.23179 [cs.AI].
- Infocomm Media Development Authority. (2026). Model AI Governance Framework for Agentic AI (Version 1.0, January 2026). Government of Singapore.
- Kamali, H. Ö., Tuna, E., Haratian, V., and Tüzün, E. (2026). Rethinking Code Review in the Age of AI: A Vision for Agentic Code Review. arXiv:2605.17548 [cs.SE].
- Matthias, A. (2004). The responsibility gap: Ascribing responsibility for the actions of learning automata. *Ethics and Information Technology*, 6(3), 175–183. <https://doi.org/10.1007/s10676-004-3422-1>
- Nian, Y., Yuan, A., Zhang, H., Li, J., Li, L., Hu, X., Wei, H., Xiao, X., Xiao, C., and Zhao, Y. (2026). Auditable Agents. arXiv:2604.05485 [cs.AI].
- Raees, M., and Papangelis, K. (2026). From Trust to Appropriate Reliance: Measurement Constructs in Human-AI Decision-Making. arXiv:2604.23896 [cs.HC].
- Sadowski, C., Söderberg, E., Church, L., Sipko, M., and Bacchelli, A. (2018). Modern Code Review: A Case Study at Google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '18)* (pp. 181–190). ACM. <https://doi.org/10.1145/3183519.3183525>
- Rhodes, J., and Kang, G. (2026). Proof of Execution: Runtime Verification for Governed AI Agent Actions. arXiv:2607.05397 [cs.AI].
- Romanchuk, O., and Bondar, R. (2026). The Responsibility Vacuum: Organizational Failure in Scaled Agent Systems. arXiv:2601.15059 [cs.AI].
- Thompson, D. F. (1980). Moral Responsibility and Public Officials: The Problem of Many Hands. *American Political Science Review*, 74(4), 905–916.
- Treude, C. (2026). Accountable Agents in Software Engineering: An Analysis of Terms of Service and a Research Roadmap. arXiv:2605.04532 [cs.SE].
- van de Poel, I., Royakkers, L., and Zwart, S. D. (2015). *Moral Responsibility and the Problem of Many Hands*. Routledge.
- Velasco, A., Wintersgill, N., Stalnaker, T., Chaparro, O., and Poshyvanyk, D. (2026). On Automated and Explainable Provenance of AI-Generated Code. arXiv:2608.02329 [cs.SE].
-

A The map, by tool

Table 1 in Section 4 is built on GitHub Copilot. This appendix presents the Accountability Event Map in full: all four tools across all nine events, on the five accountability dimensions. Each cell contains either a short quotation with its archive item identifier, or the marker NOT DOCUMENTED together with the pages searched.

Pages searched per tool, after equalisation under the single rule of Section 3.4: GitHub Copilot cloud agent, 21; Devin, 14; Cursor, 32; Claude Code, 19. The purposively selected first collection (GH-AGENT-*, DEVIN-*, CURSOR-DOC-*, CC-DOC-*) is retained and supplemented by the rule-selected set (EQ-*); three GitHub pages returned 404 and are recorded as unreachable. Platform controls applying to all four are in GH-CTRL-01 to -13 and GIT-TRAILER-01.

Candidate pages matching the rule, before the per-vendor cap: GitHub 147, Cursor 23, Claude Code 10, Devin 8. The vendors publish very different quantities of accountability-relevant documentation, and any cross-tool comparison should be read against that difference rather than around it.

NOT DOCUMENTED means no page among those searched carries a statement of the kind the row asks for. It does not mean the behaviour does not exist. Section 7 records the limit.

A.1 Task assignment

Tool	Authority	Execution and identity	Record
GitHub Copilot	"The cloud agent only responds to interactions from users with repository write access" (GH-AGENT-06)	Human assigns; "you can assign Copilot cloud agent to straightforward issues on your backlog by selecting 'Copilot' as the assignee" (GH-AGENT-01)	Issue assignment; sessions may also start "automatically, on a schedule or in response to events" via automations (GH-AGENT-03)
Devin	"Write access required — the commenter needs write or admin permission on the repository, and their GitHub account must be linked to their Devin account. Outside contributors can't trigger reviews" (DEVIN-02)	Human, by comment command	Comment
Cursor	No repository-permission gate on assignment; authority is documented over <i>configuration</i> instead: "Team members without admin permission can view PR Routing & Approval but cannot edit it" (CURSOR-DOC-08)	Human, or an automation trigger: "Choose a trigger, e.g. every hour or when a pull request is opened" (CURSOR-DOC-07)	Automation run
Claude Code	"the triggering user must have write access to the repository"; the action "runs two checks on the triggering actor before Claude starts, and the run fails when either check rejects it" (CC-DOC-02)	Human mention, or automation mode with no mention (CC-DOC-02)	Workflow run

A.2 Plan production and approval

Tool	Authority	Verification	Record
GitHub Copilot	Agent produces: "Copilot can research a repository, create an implementation plan, and make code changes on a branch" (GH-AGENT-01)	Reviewable, not gated: "You can review the diff, iterate, and create a pull request when you're ready" (GH-AGENT-01). No approval required before code is written	Plan visible in session
Devin	NOT DOCUMENTED across 14 pages	NOT DOCUMENTED across 14 pages	NOT DOCUMENTED
Cursor	Agent produces: "Plan Mode creates detailed implementation plans before writing any code" (CURSOR-DOC-01)	Enforced within the mode: "You review and edit the plan through chat or markdown files. Click to build the plan when ready" (CURSOR-DOC-01)	Plan file, saved to home directory by default; "Save to workspace" moves it into the repository (CURSOR-DOC-01)
Claude Code	Agent produces (CC-DOC-08)	Enforced within the mode: "Claude reads files and proposes a plan but makes no edits until you approve"; the tool "Presents a plan and asks the user to approve it before Claude leaves plan mode" (CC-DOC-08, CC-DOC-07)	Plan written to a file on disk before the approval call (CC-DOC-07)

This is the event where the paper's own Table 1 records an absence for GitHub Copilot. Two of the four tools document an enforced approval step here, and it is the only event in the map at which a human approval is required before the agent proceeds by default rather than by configuration. The gate exists inside a mode the user must select.

A.3 Change authorship

Tool	Execution and recorded identity	Record
GitHub Copilot	"Commits from Copilot cloud agent are authored by Copilot, with the person who started the task listed as co-author" (GH-AGENT-05); "The cloud agent's commits are authored by Copilot, with the human who started the task marked as the co-author" (GH-AGENT-06)	"Commits are signed and appear as 'Verified'; "Each commit message includes a link to the session logs" (GH-AGENT-05)
Devin	"Bug findings, flags, and automated annotations always appear as the Devin bot. When a user writes a comment or review through Devin Review, it appears under the user's GitHub identity" (DEVIN-02)	Commit; comment under human identity
Cursor	NOT DOCUMENTED for commit attribution across 32 pages. Automations run under a configurable identity: promoting an automation to Team Owned "changes the identity it runs as. It stops using your auth and starts using the team's shared automations service account" (CURSOR-DOC-07)	OIDC token with sub, owner_user_id, owner_service_account_id, team_id, cloud_agent_id, verifiable against a published key set (CURSOR-DOC-11)
Claude Code	Human is author; agent is named in a trailer. "Default commit attribution: Co-Authored-By: Claude Sonnet 5"; "The model name in the trailer reflects the active model for the session" (CC-DOC-03)	Co-authorship trailer, which "can be customized or disabled"; empty string "hides attribution entirely" (CC-DOC-03)

The direction of attribution is inverted between the first and the fourth row, and the trailer carrying it in the fourth is the one GitHub documents as a multiple-human-author mechanism (GH-CTRL-12) and Git defines no registry for (GIT-TRAILER-01).

A.4 Pull request creation

Tool	Authority	Execution	Record
GitHub Copilot	Agent, draft only: "It raises draft pull requests to propose a fix and iterates on the changes in response to feedback" (GH-AGENT-02)	Agent; "Some entry points open a pull request automatically" (GH-AGENT-03)	Pull request
Devin	Agent, via chat or review interface (DEVIN-02)	Agent	Pull request
Cursor	Agent: "It works on its own, opening a pull request when finished" (CURSOR-DOC-03)	Agent; <code>pr_created</code> is a recorded run event (CURSOR-DOC-12)	Pull request; run event log
Claude Code	Agent, on request (CC-DOC-01)	Agent: "It stages changes, writes commit messages, creates branches, and opens pull requests" (CC-DOC-01)	Pull request; attribution in description, disableable (CC-DOC-03)

A.5 Check execution

Tool	Verification	Status
GitHub Copilot	"Actions workflows triggered by pull requests raised by the agent require approval from a user with write access before they will run" (GH-AGENT-06)	Enforced; on by default, disableable (GH-CTRL-11)
Devin	"enabling branch protections to ensure checks are enforced before Devin can merge any changes" is offered as a recommendation (DEVIN-03)	Advisory
Cursor	"Cloud Agents automatically try to fix CI failures in PRs they create" (CURSOR-DOC-12); merge queue runs CI in a temporary draft PR (CURSOR-DOC-09)	Compelled at the queue, agent-performed
Claude Code	"GitHub doesn't trigger workflows on commits made with the default <code>GITHUB_TOKEN</code> " (CC-DOC-02); auto-fix responds to failures automatically (CC-DOC-06)	Not compelled; agent-performed where auto-fix is enabled

A.6 Review

Tool	Who reviews	Status	Record
GitHub Copilot	Human, advised: "You should review all outputs generated by the agent thoroughly prior to merging" (GH-AGENT-06)	Advisory	Review submission, if made
Devin	Human, advised: "After each slice is completed, it should undergo human review before merging into main" (DEVIN-04)	Advisory	Comment under human identity (DEVIN-02)
Cursor	Agent reviews; human advised. "Bugbot reviews every pull request and flags bugs, security issues, and code quality problems" (CURSOR-DOC-16); "standards for what gets merged should be the same whether code was written by hand or by an agent" (CURSOR-DOC-03)	Not compelled; agent-performed, plus advisory	Inline comments from the agent
Claude Code	Human, advised: "review Claude's changes before merging" (CC-DOC-02)	Advisory	Comment; replies under the human's account (CC-DOC-06)

A.7 Approval

Tool	Authority	Execution	Record
GitHub Copilot	Not the assigning user; the agent cannot approve its own pull request	A different human	Review submission, indistinguishable in the audit log from a comment (GH-CTRL-08)
Devin	Human, through the vendor's interface: "Leave comments, approve PRs, request changes—all within Devin Review, synced to GitHub" (DEVIN-02)	Human; recorded under the human's GitHub identity (DEVIN-02)	Review submission
Cursor	Agent, below a configured threshold: "Maximum Risk Threshold sets the highest risk level the agent may approve. If a PR exceeds the configured threshold, the agent will not approve it" (CURSOR-DOC-08)	Agent. With approvals enabled it "can also approve, request changes, and dismiss reviews" (CURSOR-DOC-07)	Review submission, produced by an agent
Claude Code	NOT DOCUMENTED as a pull request approval capability across 19 pages. Approval is documented at the tool-call level: "Claude Code requires approval before running Bash commands that can modify your system" (CC-DOC-04)	Human, per action	Permission prompt; hooks may deny, and "the hook can deny the call, but staying silent doesn't approve it" (CC-DOC-07)

A.8 Merge

Tool	Authority	Status	Record
GitHub Copilot	Human. The agent cannot approve or merge its own pull request. The Copilot app surfaces work "so users can inspect work before submitting reviews, enabling agent merge, or merging pull requests" (GH-AGENT-06)	Enforced for the cloud agent's own pull requests; agent merge is referenced as an enableable capability in the app	Merge event
Devin	Human or agent through the interface: "Merge, close, convert to draft, mark ready for review, and toggle auto-merge directly from Devin Review" (DEVIN-02)	Advisory	Merge event
Cursor	Queue: "the queue decides what runs next, what needs CI, and what can safely merge" (CURSOR-DOC-09)	Compelled, agent-performed; requires the app be permitted "to push or bypass the right merge restrictions" (CURSOR-DOC-09)	Divergent. "the original PR may appear as closed in GitHub instead of merged, even though [the product] treats it as merged across the product" (CURSOR-DOC-09)
Claude Code	NOT DOCUMENTED as a withheld capability across 19 pages	Advisory	Merge event

A.9 Deployment

Tool	Authority	Status	Record
GitHub Copilot	Named reviewers on the environment (GH-CTRL-05)	Optional; administrators may bypass protection rules and force deployments by default (GH-CTRL-05)	Environment approval
Devin	NOT DOCUMENTED across 14 pages	NOT DOCUMENTED	NOT DOCUMENTED

Tool	Authority	Status	Record
Cursor	NOT DOCUMENTED across 32 pages	NOT DOCUMENTED	NOT DOCUMENTED
Claude Code	NOT DOCUMENTED as a gate. The documentation advises "consider disabling auto-fix for repositories where a PR comment can deploy infrastructure or run privileged operations" (CC-DOC-06)	Advisory	NOT DOCUMENTED

Deployment is the thinnest row in the map. Three of four tools document nothing at this event, and the one platform mechanism that governs it is off by default with administrator bypass on.